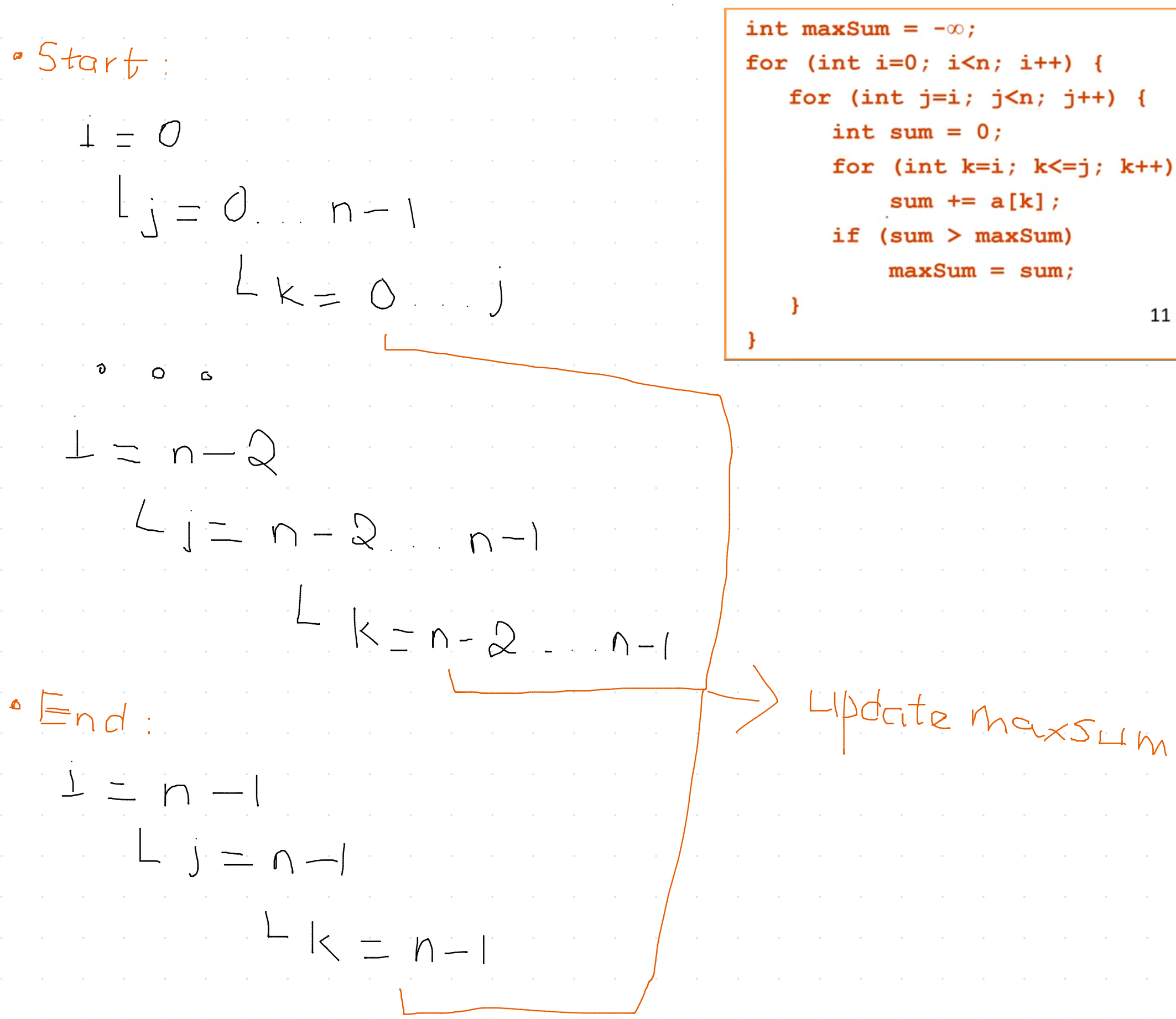
LEETCODE PROBLEMSET 53. MAXIMUM SUBARRAY SOLUTION

I. USING BRUTE FORCE - ORIGINAL SOLUTION:

1. Modeling:




```
int maxSum = -∞;
for (int i=0; i<n; i++) {
   for (int j=i; j<n; j++) {
      int sum = 0;
      for (int k=i; k<=j; k++)
          sum += a[k];
      if (sum > maxSum)
          maxSum = sum;
   }
}
```


Time complexity: O(N^3)

$$\sum_{i=0}^{n-1}\sum_{j=i}^{n-1}(j-i+1)=\sum_{i=0}^{n-1}(1+2+\ldots+(n-i))=\sum_{i=0}^{n-1}\frac{(n-i)(n-i+1)}{2}$$

$$=\frac{1}{2}\sum_{k=1}^{n}k(k+1)=\frac{1}{2}\left[\sum_{k=1}^{n}k^2+\sum_{k=1}^{n}k\right]=\frac{1}{2}\left[\frac{n(n+1)(2n+1)}{6}+\frac{n(n+1)}{2}\right]$$

$$=\frac{n^3}{6}+\frac{n^2}{2}+\frac{n}{3}$$

That may reach time limit exceed in leetcode (TLE)

Space complexity: O(1)

2. Specific:

```
int maxSum = -∞;
for (int i=0; i<n; i++) {
   for (int j=i; j<n; j++) {
      int sum = 0;
      for (int k=i; k<=j; k++)
         sum += a[k];
      if (sum > maxSum)
         maxSum = sum;
   }
}
```


For example:

We have an array

3	-1	2	5

maxSum = −∞

i = 0

j = i = 0

sum = 0

k = 0

→ initial values

Loop 1:

j = 0

k = 0

arr[k] = arr[0] = 3

Sum = sum + arr[k]

= 3

maxSum = 3

j = j + 1 = 1

k = 0

k = 1

arr[k] = arr[0] = 3

Sum = sum + arr[k]

= 3

maxSum = 3

arr[k] = −1

Sum = 2

maxSum = 3

o o o

j = 3

k = 0

k = 1

arr[k] = arr[0] = 3

sum = sum + arr[k]

= 3

maxSum = 3

k = 2

arr[k] = -1

sum = 2

maxSum = 3

k = 3

arr[k] = 2

sum = 4

maxSum = 4

arr[k] = 5

sum = 9

maxSum = 9

Continue with i = 1...(n-1)

In this example we can solve it just one i loop :D but I think that may an easy way to make sense about that problem. And also we have too many redundant calculations. Can we brainstorm a more optimized approach to solve this problem?

II. USING BRUTE FORCE - BETTER APPROACH:

maxSum = -∞

i = 0:

sum = 0

j = 0

sum = 0 + a[0]

= 3

maxSum = 3

j = 1

sum = 3 - 1

= 2

maxSum = 3

j = 2

sum = 2 + 2

= 4

maxSum = 4

j = 3

sum = 4 + 5

= 9

maxSum = 9

```
int maxSum = INT_MIN; // Option: maxSum = a[0]
for (int i=0; i<n; i++) {
    int sum = 0;
    for (int j=i; j<n; j++) {
        sum += a[j];
        if (sum > maxSum)
            maxSum = sum;
    }
}
```

14

Why is this better?

Because k-loop was removed. And now, we can reuse the recently sum for the next calculations.

$$\sum_{k=i}^{j} a[k] = a[j] - \sum_{k=i}^{j-1} a[k]$$

III. USING RECURSIVE ALGORITHM:

IV. USING DYNAMIC PROGRAMMING - KADANE'S ALGORITHM: